

Java数据结构 之



树与图

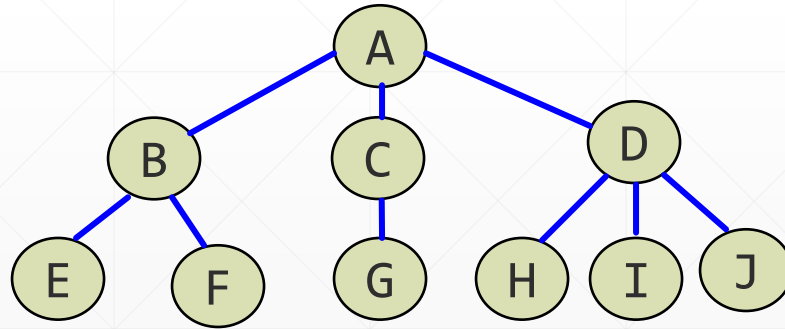
北京理工大学计算机学院
金旭亮

树 (Tree)



树

树是一种在软件开发中常用的非线性结构。树中包容多个节点，节点之间有关联，具有明显的层次关系。



二叉树

- 二叉树是由 n ($n \geq 0$) 个结点的有限集合构成，此集合或者为空集，或者由一个根结点及两棵互不相交的左右子树组成，并且左右子树都是二叉树。
- 可以使用自引用类实现二叉树：

lchild	data	rchild
--------	------	--------

```
class TreeNode<T>    {  
    private T data;  
    private TreeNode<T> lchild;  
    private TreeNode<T> rchild  
    ...  
}
```

遍历二叉树

- 访问二叉树的所有节点，称为“二叉树的遍历”，有三种典型的遍历方法：

- * Preorder traversal (前序)

若二叉树不空，则：

- (1) 访问根结点；
- (2) 前序遍历左子树；
- (3) 前序遍历右子树。

- * Postorder traversal (后序)

若二叉树不空，则：

- (1) 后序遍历左子树；
- (2) 后序遍历右子树；
- (3) 访问根结点。

- * Inorder traversal (中序)

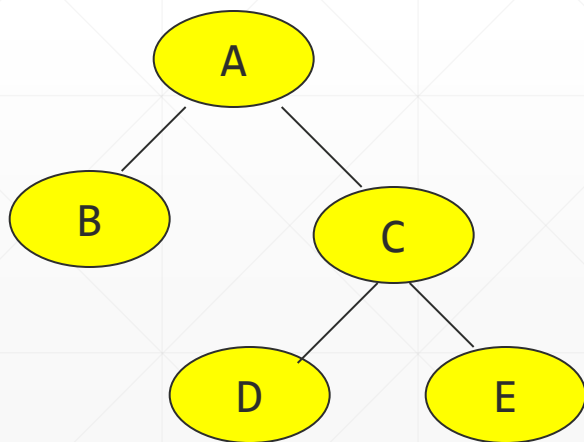
若二叉树不空，则：

- (1) 中序遍历左子树；
- (2) 访问根结点；
- (3) 中序遍历右子树。

动手动脑：二叉树编程训练

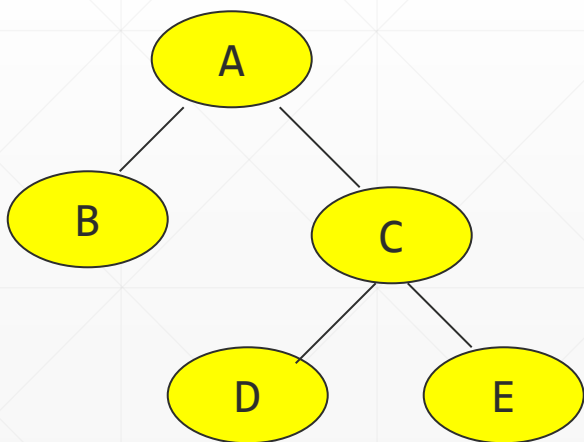
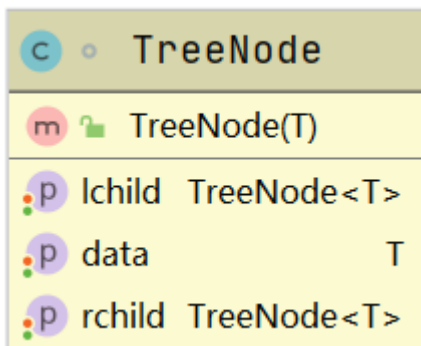


- 请使用自引用类实现一个二叉树，并且对此树进行前序、中序和后序遍历。
- 请先自己尝试着编程实现，然后再对照参考解答。



前序遍历结果：
ABCDE

二叉树示例分析-树的创建



//创建有五个节点的二叉树

```
static TreeNode<String> createTree() {  
    //创建树中所有的节点对象  
    var root = new TreeNode<String>(data: "A");  
    var bNode = new TreeNode<String>(data: "B");  
    var cNode = new TreeNode<String>(data: "C");  
    var dNode = new TreeNode<String>(data: "D");  
    var eNode = new TreeNode<String>(data: "E");  
    //建立节点之间的关联, 以便构建出一棵树  
    root.setLchild(bNode);  
    root.setRchild(cNode);  
    cNode.setLchild(dNode);  
    cNode.setRchild(eNode);  
    return root;  
}
```

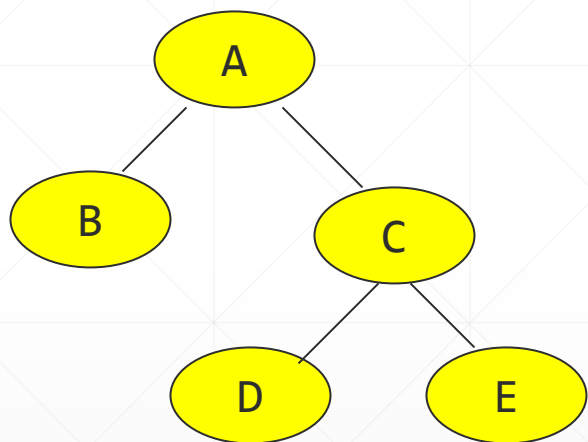
二叉树示例分析-树的遍历

//中序遍历 (递归实现)

```
static void inorderVisitTree(TreeNode<String> root) {  
    if (root != null) {  
        inorderVisitTree(root.getLchild());  
        System.out.print(root.getData() + ",");  
        inorderVisitTree(root.getRchild());  
    }  
}
```

前序和后序实现的方式是类似的.....

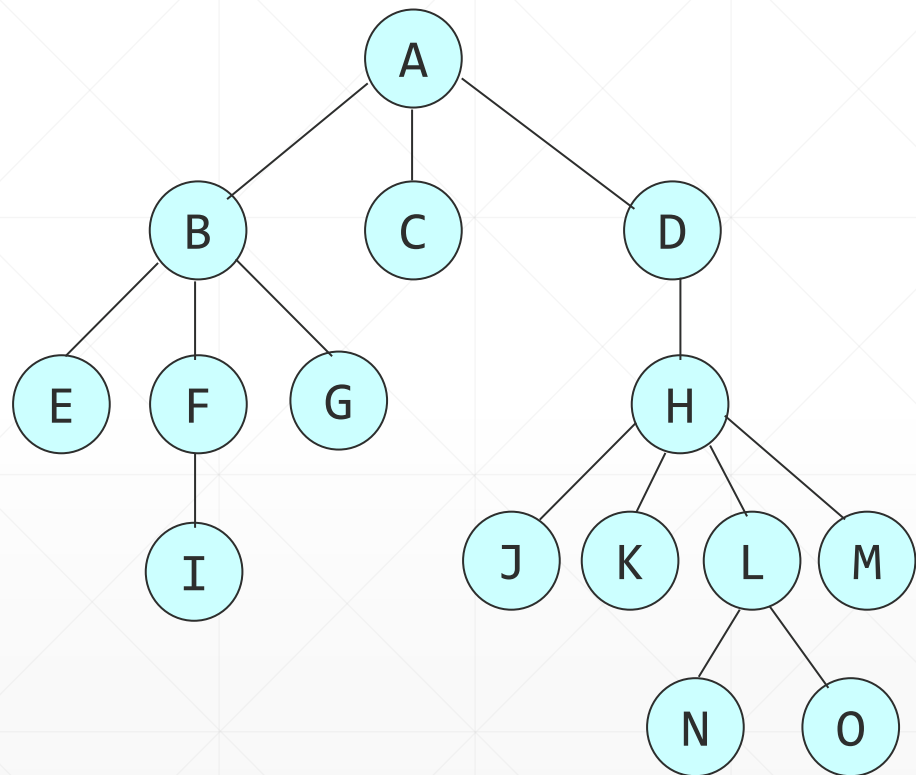
二叉树示例运行结果



```
Run: BinaryTree x
"C:\Program Files\Java\jdk-15\bin\java.exe"

---中序遍历---
B,A,D,C,E,
---前序遍历---
A,B,C,D,E,
---后序遍历---
B,D,E,C,A,
Process finished with exit code 0
```

多叉树的面向对象实现方案



对于拥有多个分支的多叉树，可以给每个节点添加一个线性表保存它的所有直接子节点，采用递归的方式实现多叉树遍历。

有两种遍历方法：

- (1) 深度优先
- (2) 广度优先

请使用Java编写代码实现多叉树的两种遍历方式。

动手动脑：多叉树编程实践



- Windows/Linux的文件系统是一种“多叉树”，请编写程序列出指定文件夹下的所有文件及子文件夹。

提示：

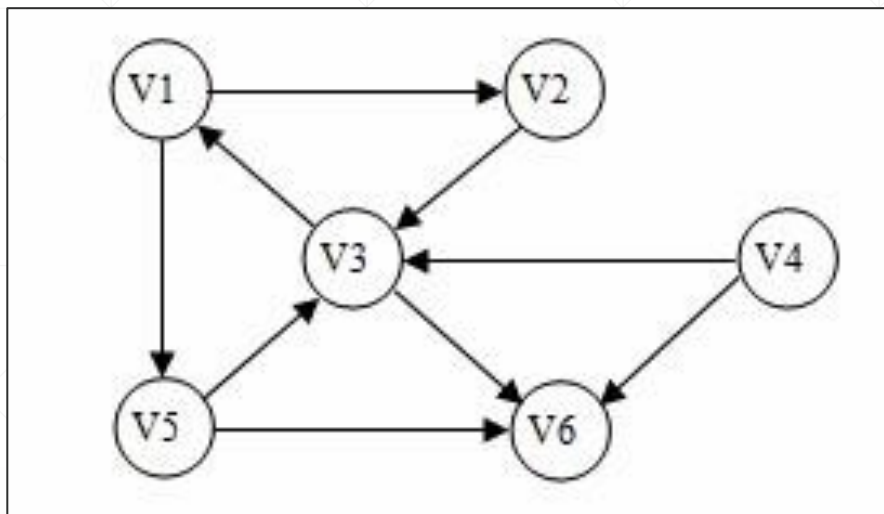
(1) 使用JDK中的File类可以列出指定文件夹下的所有文件及子文件夹，可以使用递归的方法完成这一编程任务。

(2) JDK除了经典的File类，还提供了NIO和NIO2两个包，里面包容诸多访问文件系统的组件，功能更为强大，但它们的使用比较复杂，感兴趣的同学可以自学。

(3) 遍历文件系统，往往需要耗较长的时间，在真实的应用，基本上都是使用多线程（或异常非阻塞IO操作）实现的，后面课程介绍。

图与自动机





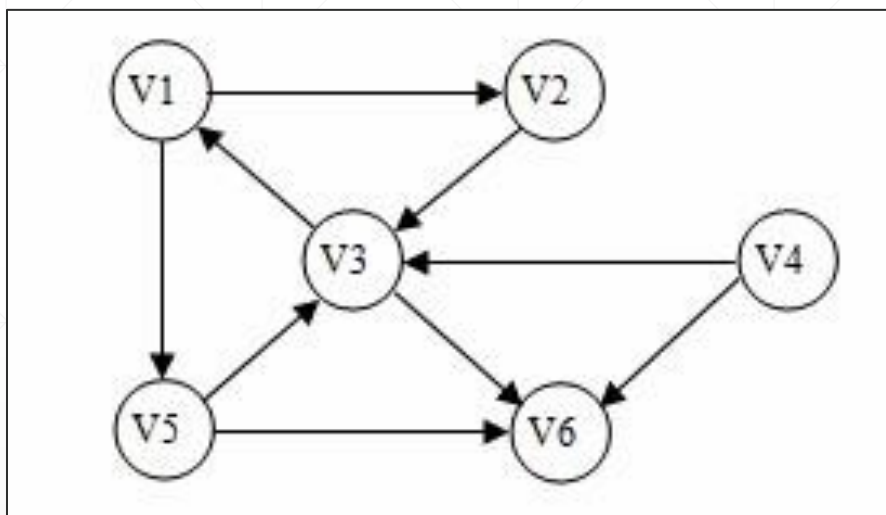
图是一种比较复杂的数据结构，它由多个节点和连接节点的边所构成。

依据边是否有方向，图可分为有向图和无向图。

《数据结构》，对于图的表示方法及相关的算法，有深入的讨论，对此感兴趣的同学，可以自行阅读相关的教材和专著，由于篇幅所限，本课程对图仅作粗略的介绍。

有向图的一种简单实现

使用自引用类和对象间的一对多关联，可以使用Java实现图这一数据结构。

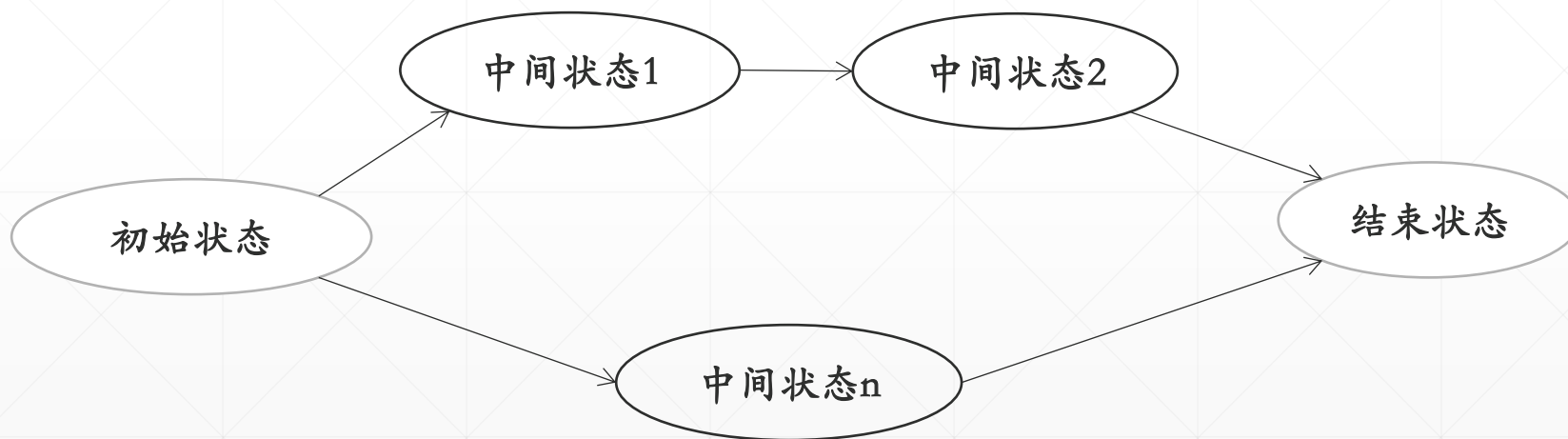


GraphNode		
f	data	T
f	inNodes	List<GraphNode<T>>
f	outNodes	List<GraphNode<T>>
m	getData()	T
m	setData(T)	void
m	getInNodes()	List<GraphNode<T>>
m	getOutNodes()	List<GraphNode<T>>
m	addNodeToInNodes(GraphNode<T>)	void
m	addNodeToOutNodes(GraphNode<T>)	void

GraphNode.java

自动机

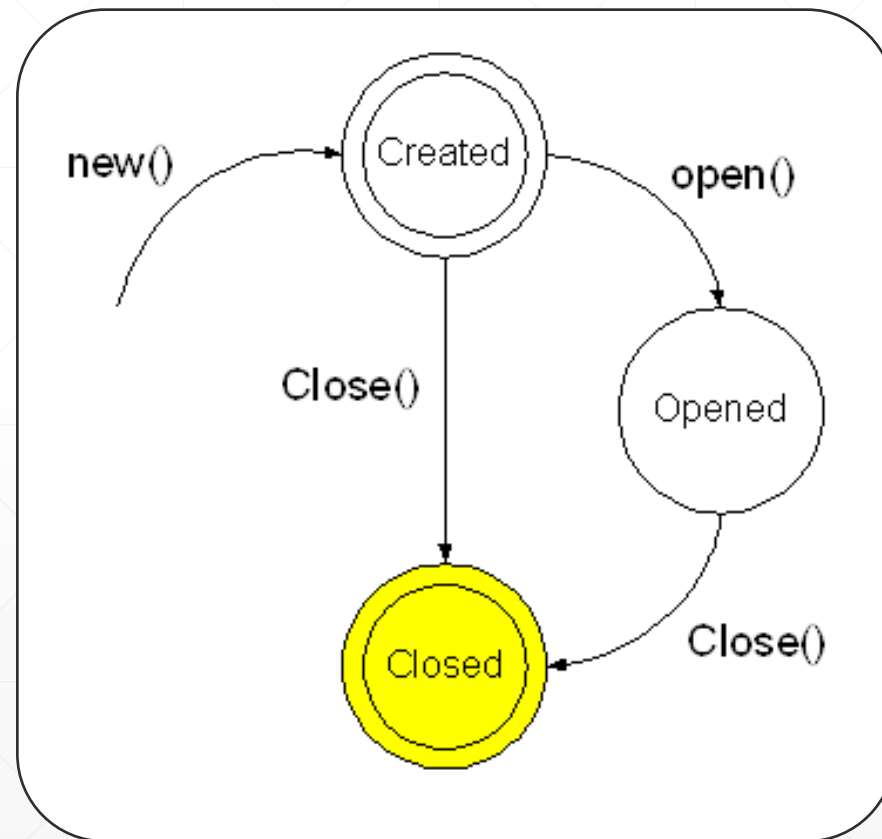
- 自动机是一种特殊的有向图，它由若干个状态组成，各个状态之间可以相互转换。



动手动脑：自动机

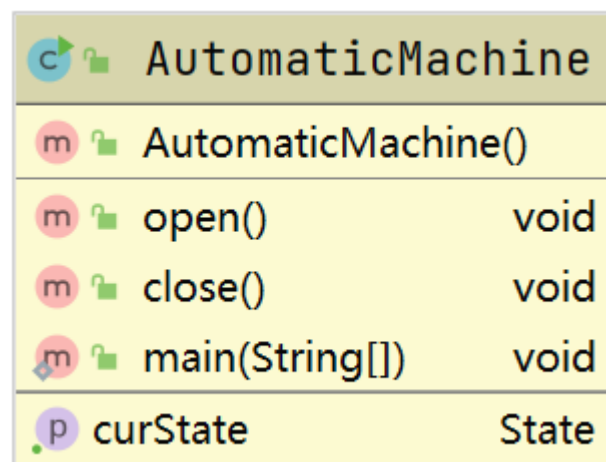
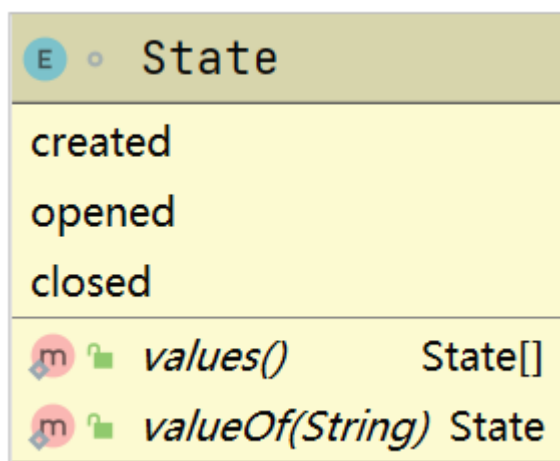


请用Java编程实现如图所示的自动机，这台自动机，应该支持状态的自动转换。



自动机参考示例类图

通常情况下，我们使用一个枚举类型表示状态机的各种状态，并在类的内部定义一个枚举类型的字段表示对象当前所处的状态，当调用特定的方法时，在方法体内部修改状态字段的值，实现状态间的转换。



AutomaticMachine.java

小结



本课程用三讲的内容，先后介绍了线性表、栈、队列、树和图的相关知识，并且给出了使用Java实现这些常用数据结构的示例，这些示例都展示了典型的数据结构面向对象实现“套路”。



数据结构与算法，是软件科学的核心内容，强烈建议同学们在学习完本课程之后，以Java作为工具，比较系统地学一学《数据结构》这门计算机专业的骨干课程，非常重要！



数据结构与算法在实际开发中有广泛的应用，JDK中就有许多组件（特别是各种集合），内部封装了特定的算法，在后面的课程中，会有相应的介绍。